

ASSIGNMENT-5.5

NAME: YEROJU ANJALI

HT.NO: 2303A51416

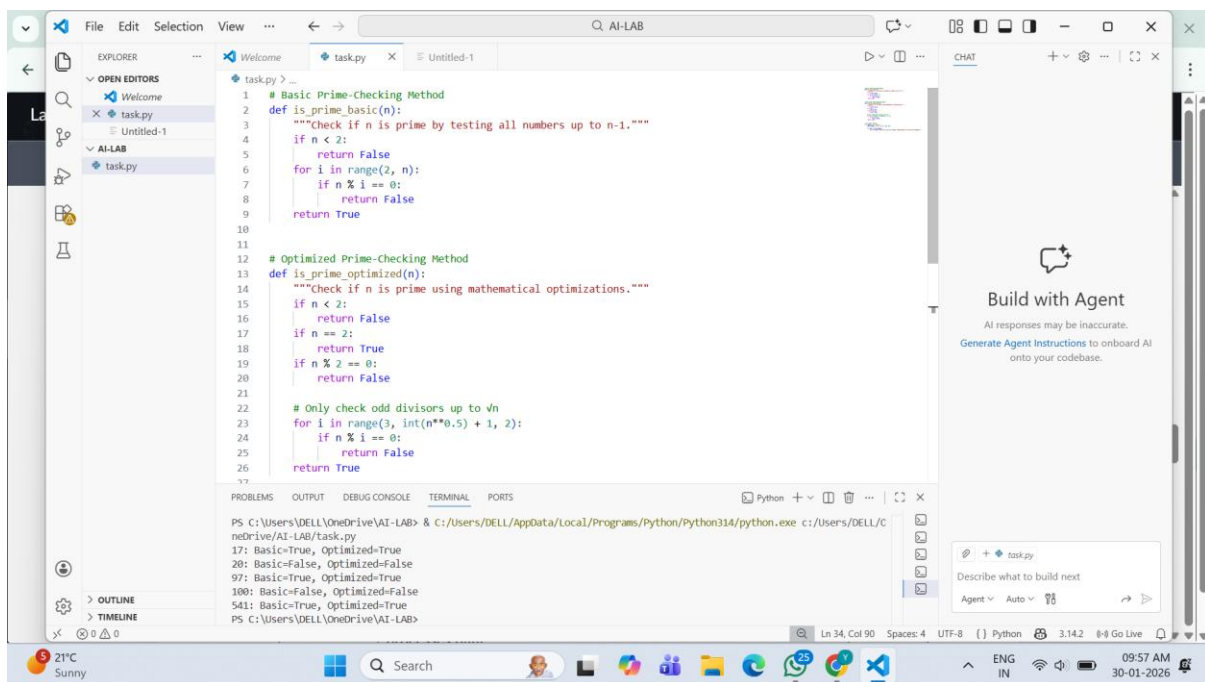
BT.NO: 29

TASK-01: (Transparency in Algorithm Optimization)

Prompt:

“Generate Python code for two prime-checking methods and Explain how the optimized version improves performance.”

CODE:



The screenshot shows a Visual Studio Code editor with a file named 'task.py'. The code defines two functions: 'is_prime_basic(n)' and 'is_prime_optimized(n)'. The basic method checks for primality by testing all numbers up to n-1. The optimized method uses mathematical insights: it checks for n < 2, n == 2, and n % 2 == 0, then only checks odd divisors up to $\sqrt{n}$. The terminal at the bottom shows the execution of these functions for various inputs, comparing the 'Basic' and 'Optimized' methods. The optimized method is significantly faster for larger numbers.

```
1 # Basic Prime-Checking Method
2 def is_prime_basic(n):
3     """Check if n is prime by testing all numbers up to n-1."""
4     if n < 2:
5         return False
6     for i in range(2, n):
7         if n % i == 0:
8             return False
9     return True
10
11
12 # Optimized Prime-Checking Method
13 def is_prime_optimized(n):
14     """Check if n is prime using mathematical optimizations."""
15     if n < 2:
16         return False
17     if n == 2:
18         return True
19     if n % 2 == 0:
20         return False
21
22     # Only check odd divisors up to sqrt(n)
23     for i in range(3, int(n**0.5) + 1, 2):
24         if n % i == 0:
25             return False
26     return True
```

Terminal Output:

```
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/C
neDrive/AI-LAB/task.py
17: Basic=True, Optimized=True
20: Basic=False, Optimized=False
97: Basic=True, Optimized=True
100: Basic=False, Optimized=False
541: Basic=True, Optimized=True
PS C:\Users\DELL\OneDrive\AI-LAB>
```

OBSERVATION:

The naive approach is simple and easy to understand but becomes inefficient for large numbers due to excessive iterations. The optimized approach leverages mathematical insights to significantly reduce computation time. By limiting checks to $\sqrt{n}$ and skipping even numbers, the optimized solution demonstrates algorithmic transparency and performance improvement. This task highlights

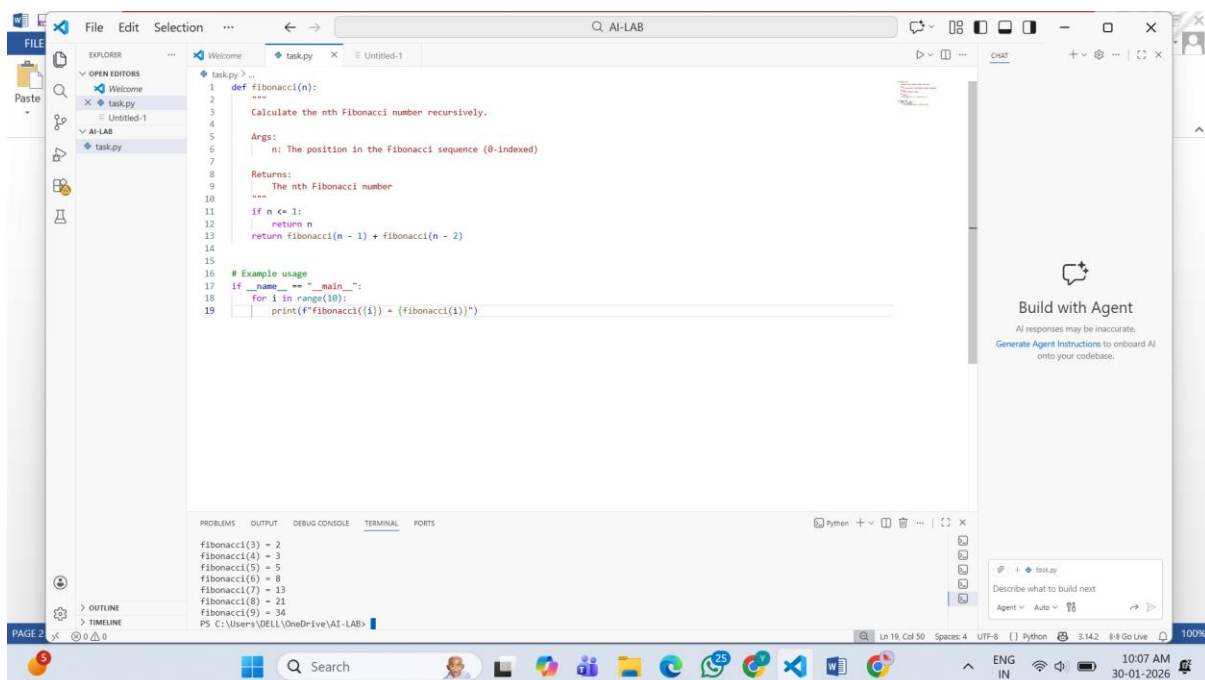
how algorithm optimization directly impacts scalability and efficiency, even for simple problems like prime checking.

TASK-02: (Transparency in Recursive Algorithms)

PROMPT:

Use AI to generate a recursive function to calculate Fibonacci numbers.

CODE:



```
1 def fibonacci(n):
2     """
3     Calculate the nth Fibonacci number recursively.
4
5     Args:
6         n: The position in the Fibonacci sequence (0-indexed)
7
8     Returns:
9         The nth Fibonacci number
10    """
11    if n <= 1:
12        return n
13    return fibonacci(n - 1) + fibonacci(n - 2)
14
15 # Example usage
16 if __name__ == "__main__":
17     for i in range(10):
18         print(f"Fibonacci({i}) = {fibonacci(i)}")
```

The screenshot shows a code editor with a Python file named 'task.py'. The code defines a recursive function 'fibonacci(n)' with a docstring explaining its purpose and arguments. The function uses a base case 'if n <= 1: return n' and a recursive call 'return fibonacci(n - 1) + fibonacci(n - 2)'. Below the function, there is an example usage block that prints the first 10 Fibonacci numbers. The terminal at the bottom shows the output of the program: 'Fibonacci(3) = 2', 'Fibonacci(4) = 3', 'Fibonacci(5) = 5', 'Fibonacci(6) = 8', 'Fibonacci(7) = 13', 'Fibonacci(8) = 21', and 'Fibonacci(9) = 34'. The status bar at the bottom indicates the file is at line 19, column 50, and the system clock shows 10:07 AM on 30-01-2026.

OBSERVATION:

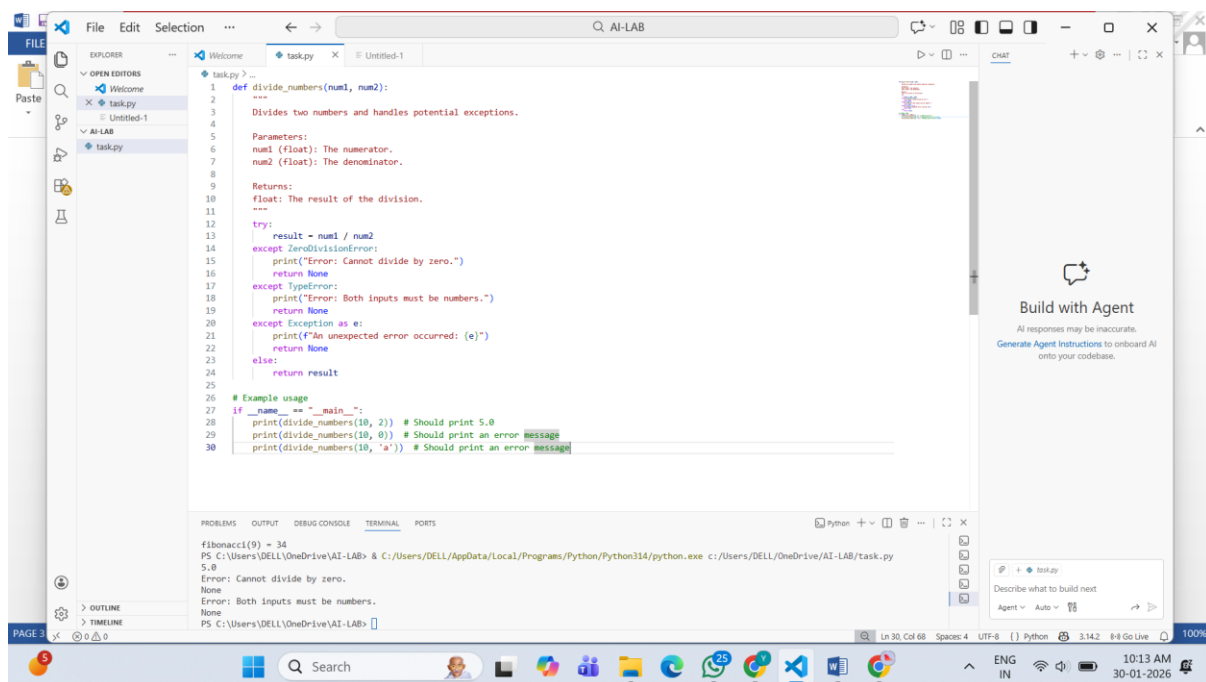
The observation from this task is that recursion becomes easier to understand and trust when the code and explanation are tightly aligned with the actual execution flow. Clear comments help reveal how each function call depends on smaller sub-problems, while explicit base cases prevent infinite recursion and anchor the computation. By walking through how recursive calls repeatedly break the Fibonacci problem into simpler cases until the base conditions are met, the explanation accurately mirrors what happens during runtime, improving transparency and reducing confusion about how and why the algorithm works.

TASK-03: (Transparency in Error Handling)

PROMPT:

“Generate code with proper error handling and clear explanations for each exception.”

CODE:



The screenshot shows a Visual Studio Code editor window with a Python file named `task.py`. The code defines a function `divide_numbers(num1, num2)` that handles various exceptions. The terminal at the bottom shows the execution of the script, which prints the Fibonacci sequence and then handles three different error cases: division by zero, non-numeric inputs, and an unexpected error.

```
1 def divide_numbers(num1, num2):
2     """
3     Divides two numbers and handles potential exceptions.
4
5     Parameters:
6     num1 (float): The numerator.
7     num2 (float): The denominator.
8
9     Returns:
10    float: The result of the division.
11    """
12    try:
13        result = num1 / num2
14    except ZeroDivisionError:
15        print("Error: Cannot divide by zero.")
16        return None
17    except TypeError:
18        print("Error: Both inputs must be numbers.")
19        return None
20    except Exception as e:
21        print(f"An unexpected error occurred: {e}")
22        return None
23    else:
24        return result
25
26 # Example usage
27 if __name__ == "__main__":
28     print(divide_numbers(10, 2)) # Should print 5.0
29     print(divide_numbers(10, 0)) # Should print an error message
30     print(divide_numbers(10, 'a')) # Should print an error message
```

Terminal Output:

```
fibonacci(9) = 34
PS C:\Users\DELL\OneDrive\AI-LAB> & C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe c:\Users\DELL\OneDrive\AI-LAB\task.py
5.0
Error: Cannot divide by zero.
None
Error: Both inputs must be numbers.
None
PS C:\Users\DELL\OneDrive\AI-LAB>
```

OBSEVATION:

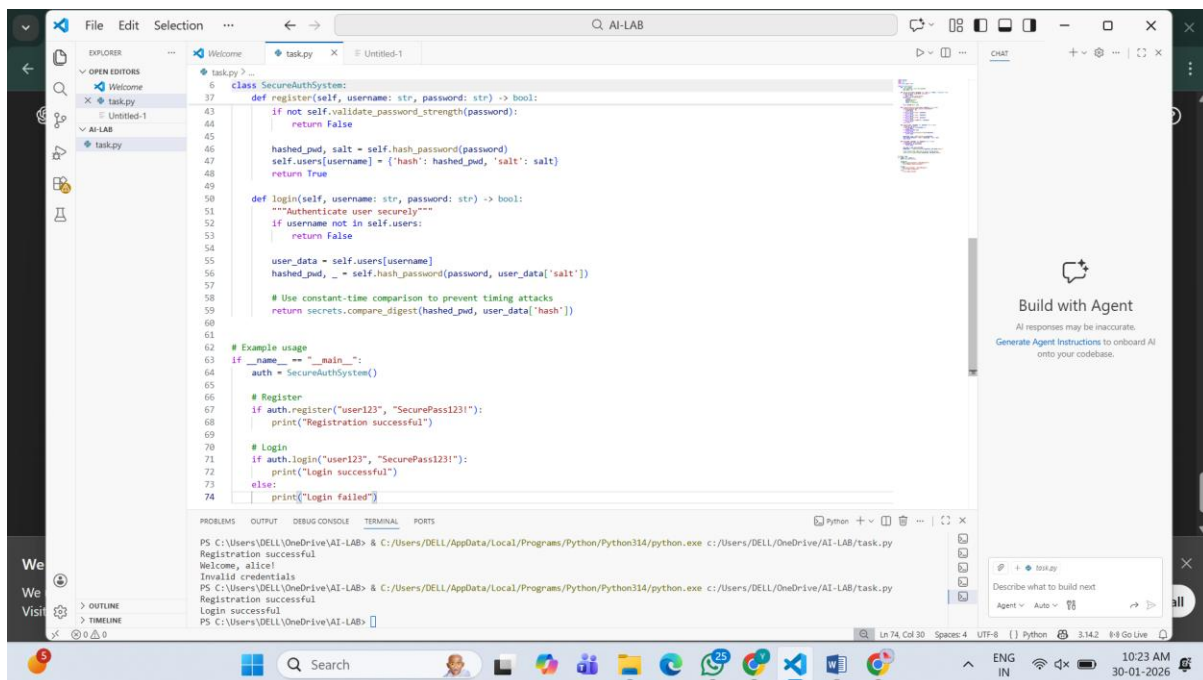
The observation from this task is that transparent error handling makes the program’s behaviour predictable and trustworthy by clearly linking each possible failure to a specific exception and explanation. When exceptions are handled with meaningful messages and well-documented scenarios (such as missing files, permission issues, or invalid data), it becomes easier to understand how the program will react at runtime. This alignment between the code, comments, and actual execution improves debugging, enhances reliability, and helps users and developers anticipate and manage errors effectively.

TASK-04: (Security in User Authentication)

PROMPT:

Check whether the AI uses secure password handling practices.

CODE:



```
6 class SecureAuthSystem:
37 def register(self, username: str, password: str) -> bool:
43     if not self.validate_password_strength(password):
44         return False
45
46     hashed_pwd, salt = self.hash_password(password)
47     self.users[username] = {'hash': hashed_pwd, 'salt': salt}
48     return True
49
50 def login(self, username: str, password: str) -> bool:
51     """Authenticate user securely"""
52     if username not in self.users:
53         return False
54
55     user_data = self.users[username]
56     hashed_pwd, _ = self.hash_password(password, user_data['salt'])
57
58     # Use constant-time comparison to prevent timing attacks
59     return secrets.compare_digest(hashed_pwd, user_data['hash'])
60
61
62 # Example usage
63 if __name__ == "__main__":
64     auth = SecureAuthSystem()
65
66     # Register
67     if auth.register("user123", "SecurePass123!"):
68         print("Registration successful")
69
70     # Login
71     if auth.login("user123", "SecurePass123!"):
72         print("Login successful")
73     else:
74         print("Login failed")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Registration successful
Welcome, alice!
Invalid credentials
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Registration successful
Login successful
PS C:\Users\DELL\OneDrive\AI-LAB>
```

OSERVATION:

The observation from this task is that AI-generated authentication systems must be carefully reviewed for security, as initial implementations may rely on insecure practices like plain-text password storage or minimal input validation. By identifying these flaws and revising the system to use password hashing and proper validation, the transparency of the authentication process improves and aligns with real-world security requirements. This demonstrates that secure authentication is not just about functionality, but about applying established best practices to protect user credentials and prevent common vulnerabilities.

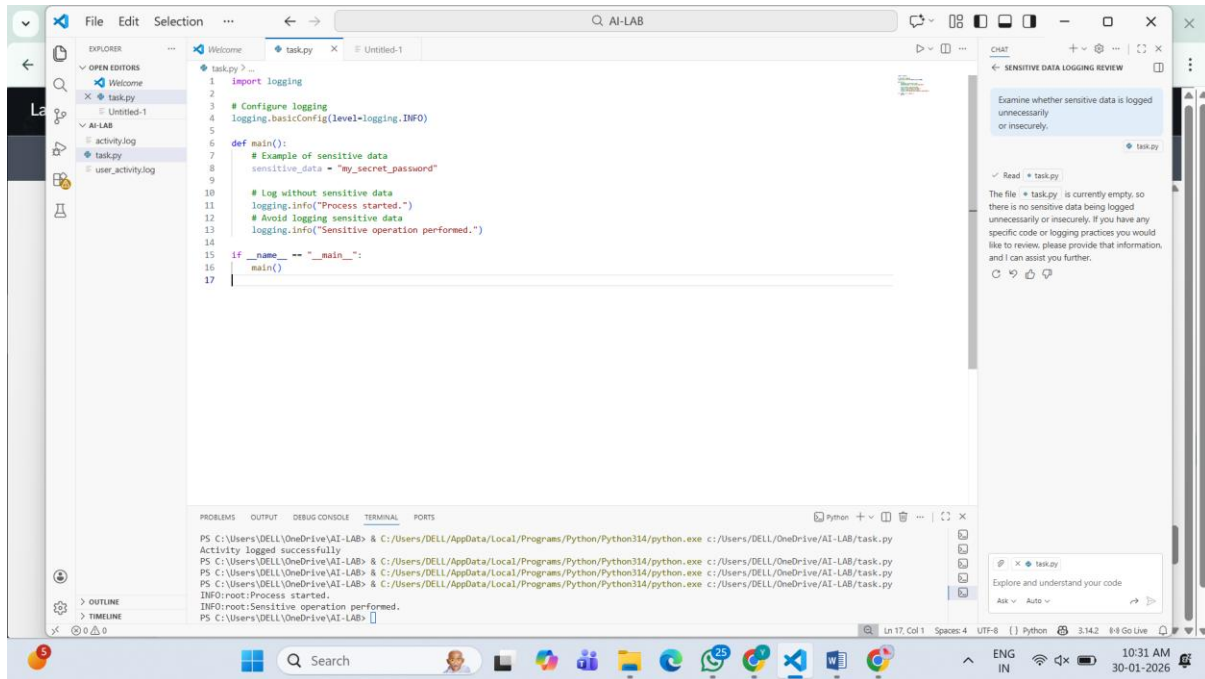
TASK-05: (Privacy in Data Logging)

PROMPT:

Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

CODE:



```
1 import logging
2
3 # Configure logging
4 logging.basicConfig(level=logging.INFO)
5
6
7 def main():
8     # Example of sensitive data
9     sensitive_data = "my_secret_password"
10
11     # Log without sensitive data
12     logging.info("Process started.")
13     # Avoid logging sensitive data
14     logging.info("Sensitive operation performed.")
15
16 if __name__ == "__main__":
17     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Activity logged successfully
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
INFO:root:Process started.
INFO:root:Sensitive operation performed.

OBSERVATION:

The observation from this task is that logging user activity without privacy considerations can easily expose sensitive information such as usernames and IP addresses, creating unnecessary privacy and security risks. By analyzing and refining the logging approach to minimize collected data and apply anonymization or masking techniques, the system becomes more privacy-aware while still remaining useful for monitoring and debugging. This highlights the importance of balancing operational needs with data protection principles and ensuring that logged information aligns with privacy best practices and real-world compliance expectations.